ESEC5023 作业一（Assignment 01）报告

姓名：__李天铸___　学号（SUSTech ID）：_12632727__　合作者：__豆包__

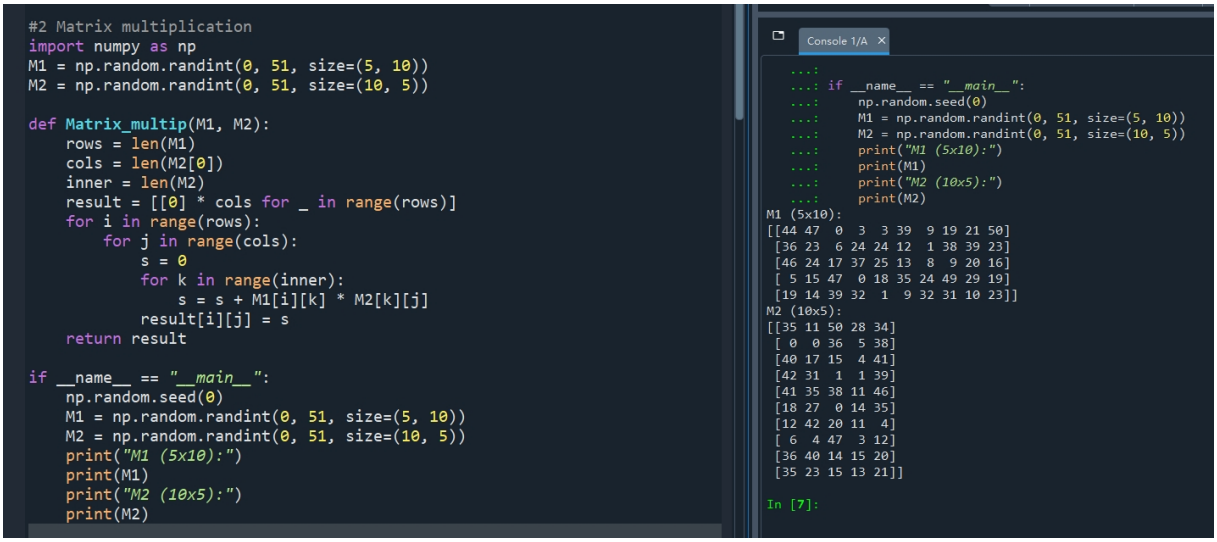
1. Flowchart

[10 points] Write a function Print_values with arguments a, b, and c to reflect the following flowchart. Here the purple parallelogram operator is to print values in the given order. Report your output with some random a, b, and c values.

```
In [4]:
   ...: import random
   ...: def Print_values(a, b, c):
   ...:     if a > b:
   ...:         if b > c:
   ...:             print(a, b, c)
   ...:         else:
   ...:             if a > c:
   ...:                 print(a, c, b)
   ...:             else:
   ...:                 print(c, a, b)
   ...:     else:
   ...:         if b > c:
   ...:             if a > c:
   ...:                 print(a, c, b)
   ...:             else:
   ...:                 print(c, a, b)
   ...:         else:
   ...:             print(c, b, a)
   ...: if __name__ == "__main__":
   ...:     random.seed(0)
   ...:     for _ in range(5):
   ...:         a = random.randint(-10, 10)
   ...:         b = random.randint(-10, 10)
   ...:         c = random.randint(-10, 10)
   ...:         print("a = %3d, b = %3d, c = %3d -> " % (a, b, c), end="")
   ...:         Print_values(a, b, c)
a =   2, b =   3, c =  -9 -> 2 -9 3
a =  -2, b =   6, c =   5 -> 5 -2 6
a =   2, b =  -1, c =   5 -> 5 2 -1
a =   1, b =   8, c =  -4 -> 1 -4 8
a =   6, b =  -6, c =  -1 -> 6 -1 -6
```

2. Matrix multiplication

2.1 [5 points] Make two matrices M1 (5 rows and 10 columns) and M2 (10 rows and 5 columns); both are filled with random integers from 0 and 50.



```
#2 Matrix multiplication
import numpy as np
M1 = np.random.randint(0, 51, size=(5, 10))
M2 = np.random.randint(0, 51, size=(10, 5))

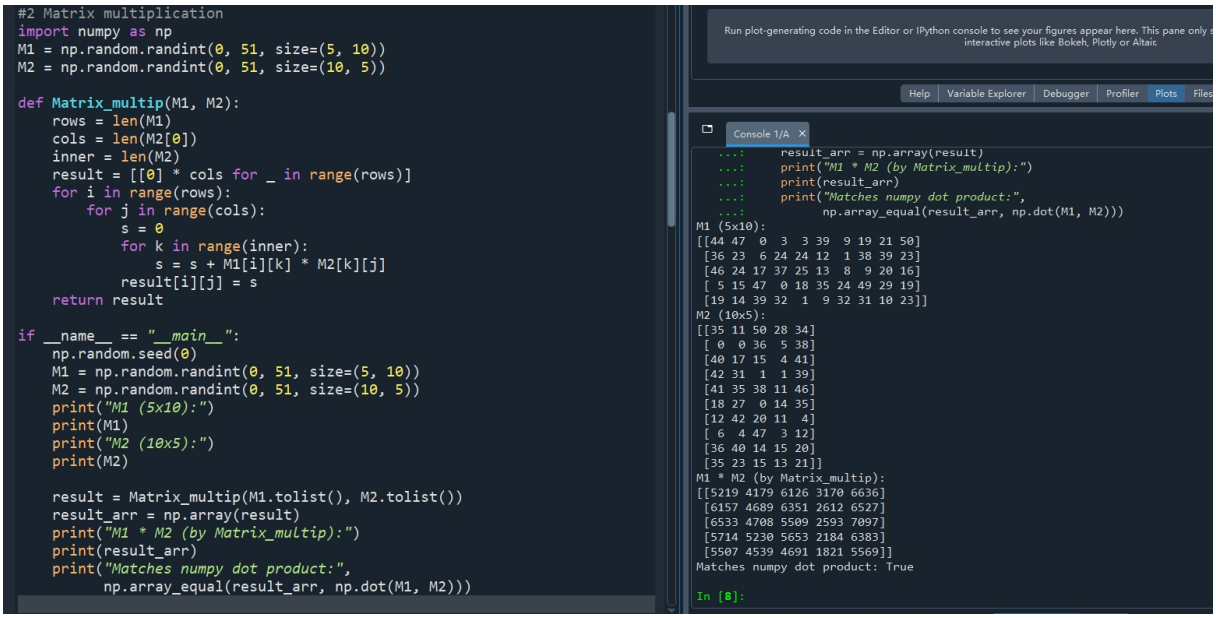
def Matrix_multip(M1, M2):
    rows = len(M1)
    cols = len(M2[0])
    inner = len(M2)
    result = [[0] * cols for _ in range(rows)]
    for i in range(rows):
        for j in range(cols):
            s = 0
            for k in range(inner):
                s = s + M1[i][k] * M2[k][j]
            result[i][j] = s
    return result

if __name__ == "__main__":
    np.random.seed(0)
    M1 = np.random.randint(0, 51, size=(5, 10))
    M2 = np.random.randint(0, 51, size=(10, 5))
    print("M1 (5x10):")
    print(M1)
    print("M2 (10x5):")
    print(M2)
```

```
Console 1/A
...:
...: if __name__ == "__main__":
...:     np.random.seed(0)
...:     M1 = np.random.randint(0, 51, size=(5, 10))
...:     M2 = np.random.randint(0, 51, size=(10, 5))
...:     print("M1 (5x10):")
...:     print(M1)
...:     print("M2 (10x5):")
...:     print(M2)
M1 (5x10):
[[44 47  0  3  3 39  9 19 21 50]
 [36 23  6 24 24 12  1 38 39 23]
 [46 24 17 37 25 13  8  9 20 16]
 [ 5 15 47  0 18 35 24 49 29 19]
 [19 14 39 32  1  9 32 31 10 23]]
M2 (10x5):
[[35 11 50 28 34]
 [ 0  0 36  5 38]
 [40 17 15  4 41]
 [42 31  1  1 39]
 [41 35 38 11 46]
 [18 27  0 14 35]
 [12 42 20 11  4]
 [ 6  4 47  3 12]
 [36 40 14 15 20]
 [35 23 15 13 21]]

In [7]:
```

2.2 [10 points] Write a function Matrix_multip to do matrix multiplication, i.e., M1 * M2. Here you are **ONLY** allowed to use for loop, * operator, and + operator.



```
#2 Matrix multiplication
import numpy as np
M1 = np.random.randint(0, 51, size=(5, 10))
M2 = np.random.randint(0, 51, size=(10, 5))

def Matrix_multip(M1, M2):
    rows = len(M1)
    cols = len(M2[0])
    inner = len(M2)
    result = [[0] * cols for _ in range(rows)]
    for i in range(rows):
        for j in range(cols):
            s = 0
            for k in range(inner):
                s = s + M1[i][k] * M2[k][j]
            result[i][j] = s
    return result

if __name__ == "__main__":
    np.random.seed(0)
    M1 = np.random.randint(0, 51, size=(5, 10))
    M2 = np.random.randint(0, 51, size=(10, 5))
    print("M1 (5x10):")
    print(M1)
    print("M2 (10x5):")
    print(M2)

    result = Matrix_multip(M1.tolist(), M2.tolist())
    result_arr = np.array(result)
    print("M1 * M2 (by Matrix_multip):")
    print(result_arr)
    print("Matches numpy dot product:",
          np.array_equal(result_arr, np.dot(M1, M2)))
```

```
Run plot-generating code in the Editor or IPython console to see your figures appear here. This pane only s
interactive plots like Bokeh, Plotly or Altair

Help | Variable Explorer | Debugger | Profiler | Plots | Files

Console 1/A
...:     result_arr = np.array(result)
...:     print("M1 * M2 (by Matrix_multip):")
...:     print(result_arr)
...:     print("Matches numpy dot product:",
...:           np.array_equal(result_arr, np.dot(M1, M2)))
M1 (5x10):
[[44 47  0  3  3 39  9 19 21 50]
 [36 23  6 24 24 12  1 38 39 23]
 [46 24 17 37 25 13  8  9 20 16]
 [ 5 15 47  0 18 35 24 49 29 19]
 [19 14 39 32  1  9 32 31 10 23]]
M2 (10x5):
[[35 11 50 28 34]
 [ 0  0 36  5 38]
 [40 17 15  4 41]
 [42 31  1  1 39]
 [41 35 38 11 46]
 [18 27  0 14 35]
 [12 42 20 11  4]
 [ 6  4 47  3 12]
 [36 40 14 15 20]
 [35 23 15 13 21]]
M1 * M2 (by Matrix_multip):
[[5219 4179 6126 3170 6636]
 [6157 4689 6351 2612 6527]
 [6533 4708 5509 2593 7097]
 [5714 5230 5653 2184 6383]
 [5507 4539 4691 1821 5569]]
Matches numpy dot product: True

In [8]:
```

3. Pascal triangle

[20 points] One of the most interesting number patterns is Pascal's triangle (named after Blaise Pascal). Write a function Pascal_triangle with an argument k to print the kth line of the Pascal triangle. Report Pascal_triangle(100) and Pascal_triangle(200).

```
In [8]:
   ...: def Pascal_triangle(k):
   ...:     line = [1]
   ...:     for _ in range(1, k):
   ...:         line = [1] + [line[i] + line[i + 1] for i in range(len(line) - 1)] + [1]
   ...:     return line
   ...:
   ...:
   ...: def print_line(k):
   ...:     line = Pascal_triangle(k)
   ...:     print("Pascal_triangle(%d): %d numbers" % (k, len(line)))
   ...:     for i in range(0, len(line), 10):
   ...:         print(line[i:i + 10])
   ...:     print("Sum check = 2^(k-1):", sum(line), "==", 2 ** (k - 1))
   ...:     print()
   ...:
   ...:
   ...: if __name__ == "__main__":
   ...:     print_line(100)
   ...:     print_line(200)
Pascal_triangle(100): 100 numbers
[1, 99, 4851, 156849, 3764376, 71523144, 1120529256, 14887031544, 171200862756, 1731030945644]
[15579278510796, 126050526132804, 924370524973896, 6186171974825304, 38000770702498296,
215337700647490344, 1130522928399324306, 5519611944537877494, 25144898858450330806,
107196674080761936594]
[428786696323047746376, 161305471473908437922 4, 571901217043857188997 6, 191462581358160885012 24,
606298174300842802538 76, 181889452290252840761628, 517685364210719623706172,
139966783656972342705742 8, 359914586546500309814767 2, 881170194648328344718912 8]
[205606378751276613767746 32, 457640004317357624192725 68, 972485009174384951409542 07,
```

```
139966783656972342705742 8, 359914586546500309814767 2, 881170194648328344718912 8]
[205606378751276613767746 32, 457640004317357624192725 68, 972485009174384951409542 07,
197443926105102399225573693, 383273503615787010261407757, 711793649572175876199757263,
126541093257275711324401 2912, 215461861492118103065872 4688, 351543037171350589212739 2912,
549849365832112460050694 7888]
[824774048748168690076042 1832, 118686997258882811498747 53368, 163901091452742930164937 07032,
217264237507124349288404 95368, 276518120463612808185242 66832, 337966591677748987781963 26128,
396743390230400985657087 30672, 447391482600239409357992 06928, 484674106150259360137824 74172,
504456722727820966674062 48628]
[504456722727820966674062 48628, 484674106150259360137824 74172, 447391482600239409357992 06928,
396743390230400985657087 30672, 337966591677748987781963 26128, 276518120463612808185242 66832,
217264237507124349288404 95368, 163901091452742930164937 07032, 118686997258882811498747 53368,
824774048748168690076042 1832]
[549849365832112460050694 7888, 351543037171350589212739 2912, 215461861492118103065872 4688,
126541093257275711324401 2912, 711793649572175876199757263, 383273503615787010261407757,
197443926105102399225573693, 972485009174384951409542 07, 457640004317357624192725 68,
205606378751276613767746 32]
[881170194648328344718912 8, 359914586546500309814767 2, 139966783656972342705742 8,
517685364210719623706172, 181889452290252840761628, 606298174300842802538 76, 191462581358160885012 24,
571901217043857188997 6, 161305471473908437922 4, 428786696323047746376]
[107196674080761936594, 251448988584503308 06, 551961194453787749 4, 113052292839932430 6,
215337700647490344, 38000770702498296, 6186171974825304, 924370524973896, 126050526132804,
15579278510796]
[1731030945644, 171200862756, 14887031544, 1120529256, 71523144, 3764376, 156849, 4851, 99, 1]
Sum check = 2^(k-1): 6338253001141147007483516026 88 == 633825300114114700748351602688

Pascal_triangle(200): 200 numbers
[1, 199, 19701, 1293699, 63391251, 2472258789, 79936367511, 2203959847089, 52895036330136,
1122550215450664]
[21328454093562616, 366461620334848584, 5741232051912627816, 82585414900589338584,
109720622653640121 2616, 135322101272822816222 64, 155620416463746238656036, 167520801252150362788556 4,
```

```
[21328454093562616, 366461620334848584, 5741232051912627816, 82585414900589338584,
109720622653640121261 6, 13532210127282281622264, 15562041646374623865 6036, 16752080125215036278855 64,
16938214348828536681954036, 16135877879673500733861 4764]
[14522290091706150660475328 76, 12378523459120956991548018324, 10015350798743319747707033007 6,
77074656146850765014962819232 4, 56521414507690561010972734103 76, 39564990155383392707680913872632,
26478108796295039735140303899376 8, 16965603043552006941404713239230 32,
10421727583896232835434323846955768, 61452255753319166029629978545842632]
[34822944926880860750123654509310824 8, 18984121589170533763777089071204933 52,
99666638343145302259829717623825900 98, 50437359403955349931489584373269471102,
24625299003107612025374326488125682949 8, 11609069530036445669105039630116393390 62,
52885761192388252492589624981641347668 38, 23298321822592662584573267221641999107962,
99324424612105561544759718155421154091838, 41003159903971783099246960571853348227656 2]
[16401263961588713239698784228741339291062 48, 63604901704697692807612358350848470603119352,
23927558260338655865720839569944246554591848, 87363410392399278393445856104215039745835352,
30974300048214289612221712618767150455341624 8, 10668925572162699755320812124242018490173226 32,
35717707350283820919987066676810235814927757 68, 11627253669347711916506428088408438467412653032,
36819636619601087735603688946626721813473401268, 11346459448081151526686034757021704069049966513 2]
[34039378344243454580058104271065112207149899539 6, 99448379868475975145659951693896111213461441238 04,
28304538885643162156841678559031970376774871215 96, 78505041814897072397277863173164144252564265448 04,
21225437231435134388893644487559194557174782880396,
55957970882874445207083244558110603832551700321044,
14389192512739143053249977172085583842656151511125 6,
36099202268801709765170995361548043675435608177034 4,
88380805554652461838866919678272796584687178640325 6,
21121514547806774778441077414638075116001512183535 44]
[49283533944882474483029180634155508604003528428249 36,
11230182325145350742854190341225599501568017133650264,
24996212272097716169578681727244076309941715555544136,
54356842559958525638607609470356165943841508430310264,
11550829043991186698204117012450685263066320541440931 1,
```

```
543568425599585256386076094703561659438415084303102 64,
11550829043991186698204117012450685263066320541440931 1,
23990183399058618527039319948936038623291588816838856 9,
48707342052634164888231346562991351144258680325097073 1,
96687708850751401942309986460863428390841857958774786 9,
18768790541616448612330762077697018452339890074350399 81,
35633500883358764756743910611279846626906168112172498 19]
[66176501640523420262524405420948286592825740779748925 21,
12023617903700734104036124365214547845738761352940297679,
21375320717690193962730887760381418392424464627449418096,
37187201796529515524203051309156714189560369968302412304,
63318749004901607514183573850726297133575765081163566896,
10553124834150267919030595641787716188929294180193927816 0,
17218256308350437131049919205022063255621479978211145384 0,
27504487349702646326222598210619659486252493991168453016 0,
43019839187996446817937910021738441760548772652853221384 0,
65891146098070507151525153324434828519321537860699237816 0]
[98836719147105760727287729986652242778982306791048856724 0,
14520456269759982131539802306681008507035672232265202407 60,
20895290729654608433191422831565353705246455163503583952 40,
29454807414091435984137306883049956427877533182288184607 60,
40675686428983411597141995219449939828973736293503503724 0,
55031811050977556866721522943961683298023290287356356115 60,
72949144881528389334956437390832929022961105729751448804 40,
94750038754169057412069855461656562983846033878872571435 60,
12059095841439698216081617967847198925216767948220145455440,
15039995937076477550393928027315045850551249912948720736560]
[18382217256426805894925912033385056039562638782492880900240,
22018260230225514753262905622406275915520083816392571627760,
25847522878960386884265150078476932596480098393156497128240,
29738547828481305339960979122548728901326564817932744007760,
```

```
2201826023022551475326290562240627591552008381639257162776O,
258475228789603868842651500784769325964800983931564971282 40,
297385478284813053399609791225487289013265648179327440077 60,
335349581895640251705942956062783538674533603266050092002 40,
370649537884655015043410635648339700640273982557213259581 60,
401536999375042932963694855285701342360296814436981031213 40,
426374339542571361806810000973476683124851256567103569226 60,
443777373809615090860149184686679813048314573161679225113 40,
452742573280516405827020885387420819372522948377066684206 60]
[452742573280516405827020885387420819372522948377066684206 60,
443777373809615090860149184686679813048314573161679225113 40,
426374339542571361806810000973476683124851256567103569226 60,
401536999375042932963694855285701342360296814436981031213 40,
370649537884655015043410635648339700640273982557213259581 60,
335349581895640251705942956062783538674533603266050092002 40,
297385478284813053399609791225487289013265648179327440077 60,
258475228789603868842651500784769325964800983931564971282 40,
220182602302255147532629056224062759155200838163925716277 60,
183822172564268058949259120333850560395626387824928809002 40]
[150399959370764775503939280273150458505512499129487207365 60,
120590958414396982160816179678471989252167679482201454554 40,
947500387541690574120698554616565629838460338788725714356 0,
729491448815283893349564373908329290229611057297514488044 0,
550318110509775568667215229439616832980232902873563561156 0,
406756864289834115971419952194499398289737362993503501724 0,
294548074140914359841373068830499564278775331822881846076 0,
208952907296546084331914228315653537052464551635035839524 0,
145204562697599821315398023066810085070356722322652024076 0,
988367191471057607272877299866522427789823067910488567240]
[658911460980705071515251533244348285193215378606992378160,
430198391879964468179379100217384417605487726528532213840,
```


```
[658911460980705071515251533244348285193215378606992378160,
430198391879964468179379100217384417605487726528532213840,
```


```
275044873497026463262225982106196594862524939911684530160,
172182563083504371310499192050220632556214799782111453840,
105531248341502679190305956417877161889292941801939278160,
633187490049016075141835738507262971335757650811635668 96,
371872017965295155242030513091567141895603699683024123 04,
213753207176901939627308877603814183924244646274494180 96,
120236179037007341040361243652145478457387613529402976 79,
661765016405234202625244054209482865928257407797489252 1]
[356335008833587647567439106112798466269061681121724981 9,
187687905416164486123307620776970184523398900743503998 1,
966877088507514019423099864608634283908418579587747869,
487073420526341648882313465629913511442586803250970731,
239901833990586185270393199489360386232915888168388569,
115508290439911866982041170124506852630663205414409311,
543568425599585256386076094703561659438415084303102 64,
249962122720977161695786817272440763099417155555441 36,
112301823251453507428541903412255995015680171336502 64,
492835339448824744830291806341555086040035284282493 6]
[211215145478067747784410774146380751160015121835354 4,
883808055546524618388669196782727965846871786403256,
360992022688017097651709953615480436754356081770344,
143891925127391430532499771720855838426561515111256,
559579708828744452070832445581106038325517003210 44,
212254372314351343888936444875591945571747828803 96, 785050418148970723972778631731641442525642654480 4,
283045388856431621568416785590319703767748712159 6, 994483798684759751456599516938961121346144123804,
340393783442434545800581042710651122071498995396]
[113464594480811515266860347570217040690499665132, 368196366196010877356036889466267218134734012 68,
116272536693477119165064280884084384674126530 32, 357177073502838209199870666768102358149277576 8,
106689255721626997553208121242420184901732263 2, 309743000482142896122217126187671504553416248,
```

```
21225437231435134388893644487559194557174782880396, 78505041814897072397277863173164144252564265448804,
28304538885643162156841678559031970376774871215 96, 994483798684759751456599516938961121346144123804,
34039378344243454580058104271065112207149899539 6]
[11346459448081151526686034757021704069049966513 2, 36819636619601087735603688946626721813473401268,
11627253669347711916506428088408438467412653032, 35717707350283820919987066676810235814927757 68,
10668925572162699755320812124242018490173226 32, 30974300048214289612221712618767150455341624 8,
87363410392399278393445856104215039745835352, 23927558260338655865720839569944246554591848,
63604901704697692807612358350484706031193 52, 16401263961588713239698784228741339291062 48]
[41003159903971783099246960571853348227656 2, 99324424612105561544759718155421154091838,
23298321822592662584573267221641999107962, 52885761192388252492589624981641347668 38,
11609069530036445669105039630116393390 62, 24625299003107612025374326488125682 9498,
50437359403955349931489584373269471102, 99666638343145302259829717623825900 98,
18984121589170533763777089071204933 52, 34822944926880860750123654509310824 8]
[61452255753319166029629978545842632, 10421727583896232835434323846955768,
16965603043552006941404713239230 32, 26478108796295039735140303899376 8,
39564990155383392707680913872632, 56521414507690561010972734103 76, 77074656146850765014962819232 4,
10015350798743319747707033007 6, 12378523459120956991548018324, 14522290091706150660475328 76]
[16135877879673500733861476 4, 16938214348828536681954036, 16752080125215036278855 64,
15562041646374623865603 6, 13532210127282281622264, 10972062265364012126 16, 82585414900589338584,
5741232051912627816, 366461620334848584, 21328454093562616]
[1122550215450664, 52895036330136, 2203959847089, 79936367511, 2472258789, 63391251, 1293699, 19701,
199, 1]
Sum check = 2^(k-1): 80346902212949513777098104617058130126110149689139641765 0688 ==
80346902212949513777098104617058130126110149689139641765 0688


In [9]:
```

4. Add or double

[20 points] If you start with 1 RMB and, with each move, you can either double your money or add another 1 RMB, what is the smallest number of moves you have to make to get to exactly x RMB? Here x is an integer randomly selected from 1 to 100. Write a function Least_moves to print your results. For example, Least_moves(2) should print 1, and Least_moves(5) should print 3.

```python
#4 Add or double
import random


def Least_moves(x):
    moves = 0
    while x > 1:
        if x % 2 == 0:
            x = x // 2
        else:
            x = x - 1
        moves = moves + 1
    return moves


if __name__ == "__main__":
    print("Least_moves(2) =", Least_moves(2))
    print("Least_moves(5) =", Least_moves(5))
    random.seed(1)
    x = random.randint(1, 100)
    print("Random x =", x, "-> Least_moves(%d) = %d" % (x, Least_moves(x))
```

```
In [9]:
   ...: import random
   ...:
   ...:
   ...: def Least_moves(x):
   ...:     moves = 0
   ...:     while x > 1:
   ...:         if x % 2 == 0:
   ...:             x = x // 2
   ...:         else:
   ...:             x = x - 1
   ...:         moves = moves + 1
   ...:     return moves
   ...:
   ...:
   ...: if __name__ == "__main__":
   ...:     print("Least_moves(2) =", Least_moves(2))
   ...:     print("Least_moves(5) =", Least_moves(5))
   ...:     random.seed(1)
   ...:     x = random.randint(1, 100)
   ...:     print("Random x =", x, "-> Least_moves(%d) = %d" % (x, Least_moves(x)))
Least_moves(2) = 1
Least_moves(5) = 3
Random x = 18 -> Least_moves(18) = 5

In [10]:
```

5. Dynamic programming

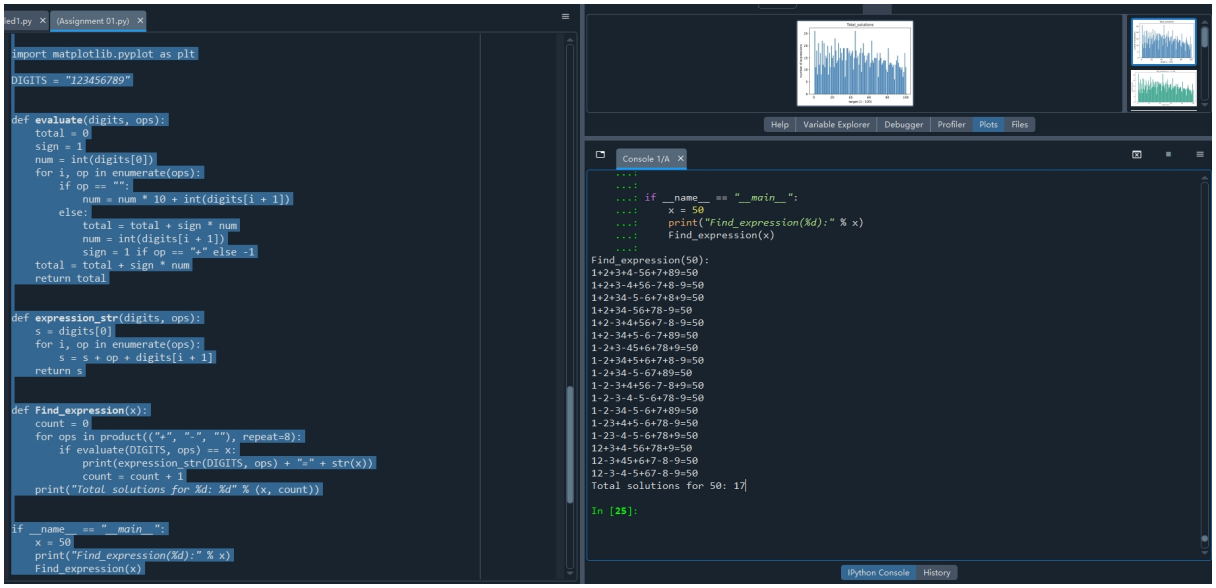
Insert + or - operation anywhere between the digits 123456789 in a way that the expression evaluates to an integer number. You may join digits together to form a bigger number. However, the digits must stay in the original order.

5.1 [30 points] Write a function Find_expression, which should be able to print every possible solution that makes the expression evaluate to a random integer from 1 to 100. For example, Find_expression(50) should print lines include:

$$1-2+34+5+6+7+8-9=501-2+34+5+6+7+8-9=50$$

and

$$1+2+34-56+78-9=501+2+34-56+78-9=50$$



```
import matplotlib.pyplot as plt

DIGITS = "123456789"

def evaluate(digits, ops):
    total = 0
    sign = 1
    num = int(digits[0])
    for i, op in enumerate(ops):
        if op == "":
            num = num * 10 + int(digits[i + 1])
        else:
            total = total + sign * num
            num = int(digits[i + 1])
            sign = 1 if op == "+" else -1
    total = total + sign * num
    return total

def expression_str(digits, ops):
    s = digits[0]
    for i, op in enumerate(ops):
        s = s + op + digits[i + 1]
    return s

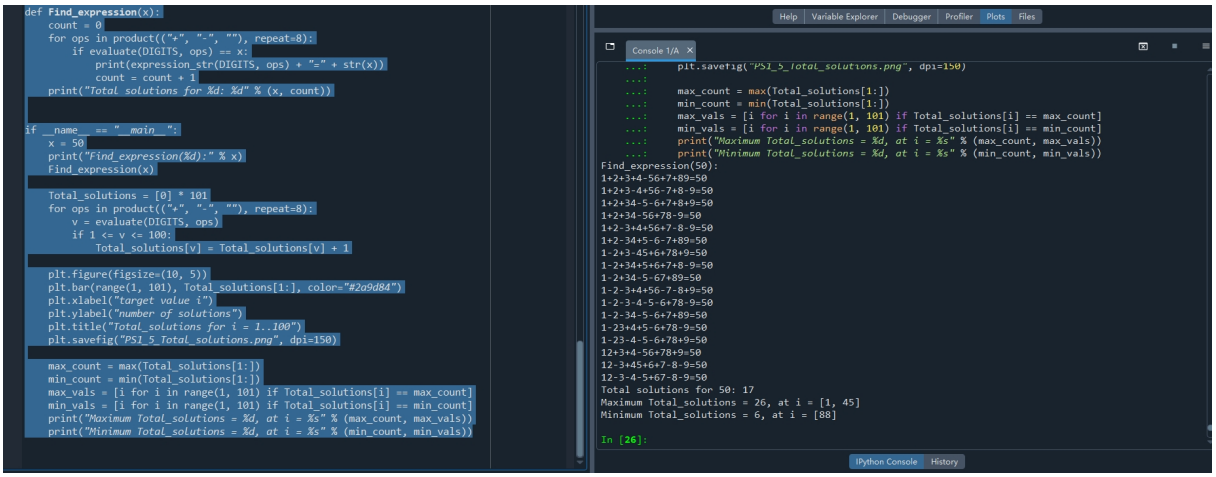
def Find_expression(x):
    count = 0
    for ops in product(("+", "-", ""), repeat=8):
        if evaluate(DIGITS, ops) == x:
            print(expression_str(DIGITS, ops) + "=" + str(x))
            count = count + 1
    print("Total solutions for %d: %d" % (x, count))

if __name__ == "__main__":
    x = 50
    print("Find_expression(%d):" % x)
    Find_expression(x)
```

```
...:
...: if __name__ == "__main__":
...:     x = 50
...:     print("Find_expression(%d):" % x)
...:     Find_expression(x)
...:
Find_expression(50):
1+2+3+4-56+7+89=50
1+2+3-4+56-7+8-9=50
1+2+34-5-6+7+8+9=50
1+2+34-56+78-9=50
1+2-3+4+56+7-8-9=50
1+2-34+5-6-7+89=50
1-2+3-45+6+78+9=50
1-2+34+5+6+7+8-9=50
1-2+34-5-67+89=50
1-2-3+4+56-7-8+9=50
1-2-3-4-5-6+78-9=50
1-2-34-5-6+7+89=50
1-23+4+5-6+78-9=50
1-23-4-5-6+78+9=50
12+3+4-56+78+9=50
12-3+45+6+7-8-9=50
12-3-4-5+67-8-9=50
Total solutions for 50: 17

In [25]:
```

5.2 [5 points] Count the total number of suitable solutions for any integer i from 1 to 100, assign the count to a list called Total_solutions. Plot the list Total_solutions, so which number(s) yields the maximum and minimum of Total_solutions?



```
def Find_expression(x):
    count = 0
    for ops in product(("+", "-", ""), repeat=8):
        if evaluate(DIGITS, ops) == x:
            print(expression_str(DIGITS, ops) + "=" + str(x))
            count = count + 1
    print("Total solutions for %d: %d" % (x, count))

if __name__ == "__main__":
    x = 50
    print("Find_expression(%d):" % x)
    Find_expression(x)

    Total_solutions = [0] * 101
    for ops in product(("+", "-", ""), repeat=8):
        v = evaluate(DIGITS, ops)
        if 1 <= v <= 100:
            Total_solutions[v] = Total_solutions[v] + 1

    plt.figure(figsize=(10, 5))
    plt.bar(range(1, 101), Total_solutions[1:], color="#2a9d84")
    plt.xlabel("target value i")
    plt.ylabel("number of solutions")
    plt.title("Total_solutions for i = 1..100")
    plt.savefig("PS1_5_Total_solutions.png", dpi=150)

    max_count = max(Total_solutions[1:])
    min_count = min(Total_solutions[1:])
    max_vals = [i for i in range(1, 101) if Total_solutions[i] == max_count]
    min_vals = [i for i in range(1, 101) if Total_solutions[i] == min_count]
    print("Maximum Total_solutions = %d, at i = %s" % (max_count, max_vals))
    print("Minimum Total_solutions = %d, at i = %s" % (min_count, min_vals))
```

```
...:     plt.savefig("PS1_5_Total_solutions.png", dpi=150)
...:
...:     max_count = max(Total_solutions[1:])
...:     min_count = min(Total_solutions[1:])
...:     max_vals = [i for i in range(1, 101) if Total_solutions[i] == max_count]
...:     min_vals = [i for i in range(1, 101) if Total_solutions[i] == min_count]
...:     print("Maximum Total_solutions = %d, at i = %s" % (max_count, max_vals))
...:     print("Minimum Total_solutions = %d, at i = %s" % (min_count, min_vals))
Find_expression(50):
1+2+3+4-56+7+89=50
1+2+3-4+56-7+8-9=50
1+2+34-5-6+7+8+9=50
1+2+34-56+78-9=50
1+2-3+4+56+7-8-9=50
1+2-34+5-6-7+89=50
1-2+3-45+6+78+9=50
1-2+34+5+6+7+8-9=50
1-2+34-5-67+89=50
1-2-3+4+56-7-8+9=50
1-2-3-4-5-6+78-9=50
1-2-34-5-6+7+89=50
1-23+4+5-6+78-9=50
1-23-4-5-6+78+9=50
12+3+4-56+78+9=50
12-3+45+6+7-8-9=50
12-3-4-5+67-8-9=50
Total solutions for 50: 17
Maximum Total_solutions = 26, at i = [1, 45]
Minimum Total_solutions = 6, at i = [88]

In [26]:
```

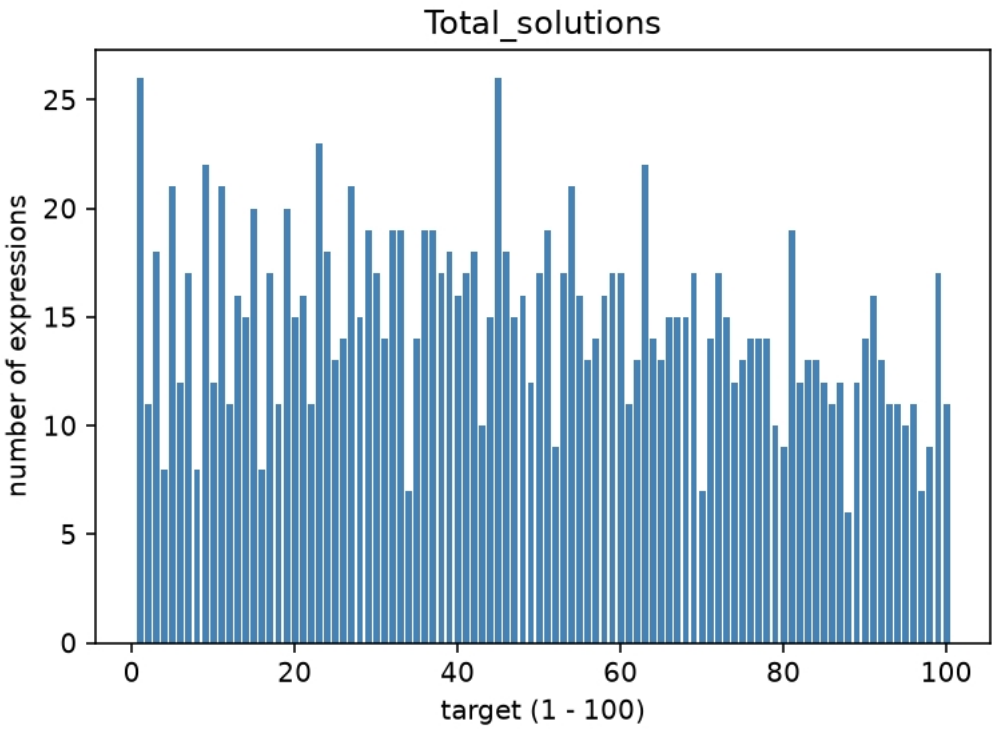


图 1 目标值 i（1~100）对应的表达式总数 Total_solutions

结论：Total_solutions 的最大值为 26，出现在 i = [1, 45] 处；最小值为 6，出现在 i = [88] 处。